

AVL TREE, RED-BLACK TREE, B TREE AND B+ TREE

Submitted by
Meera K.N
IMSc 3rd Year
Department of Computer Science

AVL Tree

An AVL tree is a self-balancing binary search tree where the height difference between left and right subtree is atmost 1.

Balancing factor = Height of left subtree - Height of right subtree
for a AVL tree:

$$-1 \leq \text{Balance Factor} \leq 1$$

Rules for AVL Tree Construction

1. The tree must satisfy the Binary Search Tree (BST) property.
2. Calculate the balance factor:

$$BF = h_{left} - h_{right}$$

3. For every node:

$$-1 \leq BF \leq 1$$

4. If $BF = 2$ or $BF = -2$, the subtree is unbalanced.

5. Apply the appropriate rotation:

- LL case $\rightarrow$ Right Rotation
- RR case $\rightarrow$ Left Rotation
- LR case $\rightarrow$ Left Rotation + Right Rotation
- RL case $\rightarrow$ Right Rotation + Left Rotation

6. Recalculate the heights after rotation.

Rules for AVL Tree Deletion

- (a) First, delete the node using the normal Binary Search Tree (BST) deletion procedure.
- (b) Update the height of each ancestor.
- (c) Calculate $BF = \text{height}(\text{left}) - \text{height}(\text{right})$.
- (d) If $BF = +2 \rightarrow$ check LL/LR.
- (e) If $BF = -2 \rightarrow$ check RR/RL.
- (f) Perform the required rotation.
- (g) Continue checking ancestors up to the root.
- (h) The final tree must satisfy $-1 \leq BF \leq 1$.

AVL Tree Insertion Example

The elements are inserted in the following order:

3, 6, 8, 10, 11, 15, 20, 35

The balance factor of a node is defined as:

$$BF = h_{left} - h_{right}$$

For an AVL tree:

$$-1 \leq BF \leq 1$$

If the balance factor becomes 2 or -2 , the tree is unbalanced and a rotation is required.

Step 1: Insert 3

The first element becomes the root.

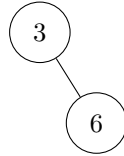


$$BF(3) = 0$$

Therefore, the tree is balanced.

Step 2: Insert 6

Since $6 > 3$, 6 is inserted to the right of 3.



Balance factors:

$$BF(6) = 0$$

$$BF(3) = 0 - 1 = -1$$

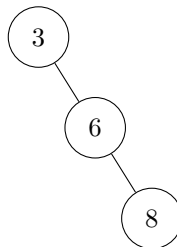
Therefore, the tree is balanced.

Step 3: Insert 8

Since

$$8 > 6 > 3$$

8 is inserted to the right of 6.



Balance factors:

$$BF(8) = 0$$

$$BF(6) = 0 - 1 = -1$$

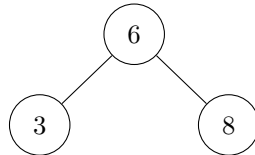
$$BF(3) = 0 - 2 = -2$$

Node 3 becomes unbalanced.

This is an **RR case**.

Therefore, perform a **left rotation at node 3**.

After rotation:



Balance factors:

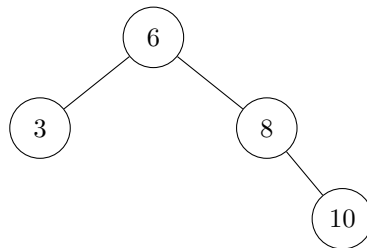
$$BF(3) = 0, \quad BF(8) = 0, \quad BF(6) = 0$$

Step 4: Insert 10

Since

$$10 > 6, \quad 10 > 8$$

10 is inserted to the right of 8.



Balance factors:

$$BF(10) = 0$$

$$BF(8) = 0 - 1 = -1$$

$$BF(6) = 1 - 2 = -1$$

Therefore, the tree remains balanced.

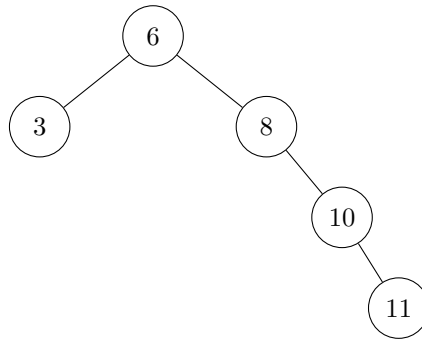
No rotation is required.

Step 5: Insert 11

Since

$$11 > 6, \quad 11 > 8, \quad 11 > 10$$

11 is inserted to the right of 10.



Balance factors:

$$BF(11) = 0$$

$$BF(10) = 0 - 1 = -1$$

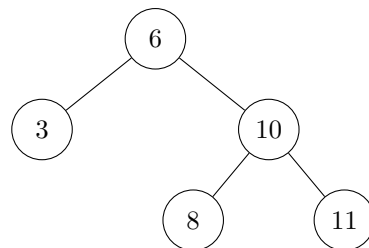
$$BF(8) = 0 - 2 = -2$$

Node 8 becomes unbalanced.

This is an **RR** case.

Perform a **left rotation at node 8**.

After rotation:



This is the tree shown in your image.

Now:

$$BF(3) = 0$$

$$BF(8) = 0$$

$$BF(11) = 0$$

$$BF(10) = 1 - 1 = 0$$

$$BF(6) = 1 - 2 = -1$$

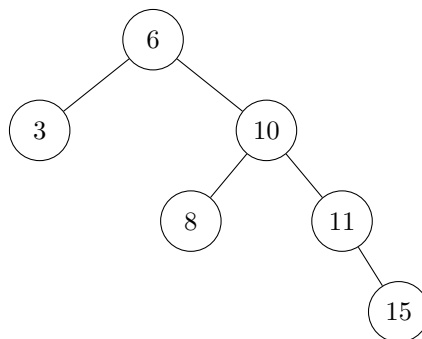
Therefore, ****node 6 is balanced**** and no additional rotation is required.

Step 6: Insert 15

Since

$$15 > 6, \quad 15 > 10, \quad 15 > 11$$

15 is inserted to the right of 11.



Balance factors:

$$BF(15) = 0$$

$$BF(11) = 0 - 1 = -1$$

$$BF(10) = 1 - 2 = -1$$

$$BF(6) = 1 - 3 = -2$$

However, the first unbalanced node while moving upward from the inserted node is **10**, because:

$$BF(10) = 1 - 2 = -1$$

Actually, the first node that becomes unbalanced is **10** only if its right subtree has height **2**. Therefore:

$$BF(10) = 1 - 2 = -1$$

and

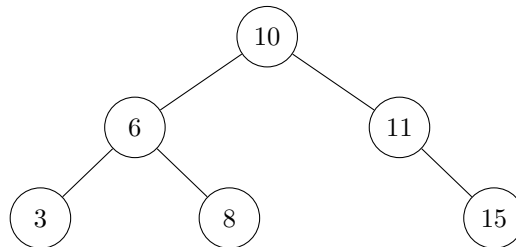
$$BF(6) = 1 - 3 = -2$$

Thus node **6** is the unbalanced node.

This is an **RR case**.

Perform a **left rotation at node 6**.

After rotation:



Balance factors:

$$BF(3) = 0, \quad BF(8) = 0, \quad BF(15) = 0$$

$$BF(6) = 0$$

$$BF(11) = -1$$

$$BF(10) = 2 - 2 = 0$$

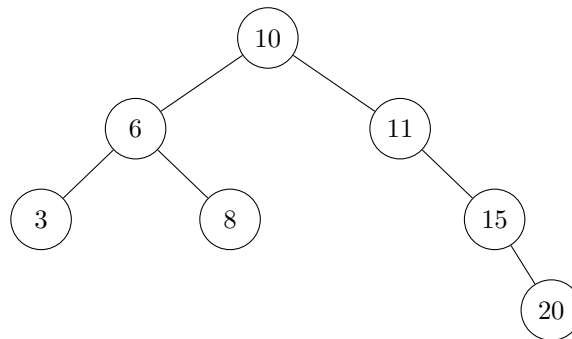
The tree is balanced.

Step 7: Insert 20

Since

$$20 > 10, \quad 20 > 11, \quad 20 > 15$$

20 is inserted to the right of 15.



Balance factors:

$$BF(20) = 0$$

$$BF(15) = 0 - 1 = -1$$

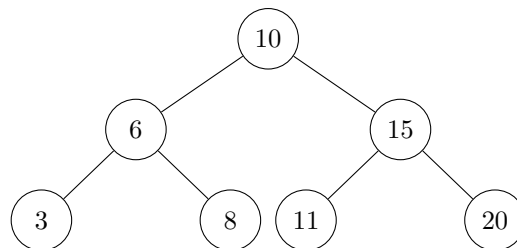
$$BF(11) = 0 - 2 = -2$$

Node 11 becomes unbalanced.

This is an **RR case**.

Perform a **left rotation at node 11**.

After rotation:



Balance factors:

$$BF(11) = 0, \quad BF(20) = 0$$

$$BF(15) = 0$$

$$BF(6) = 0$$

$$BF(10) = 2 - 2 = 0$$

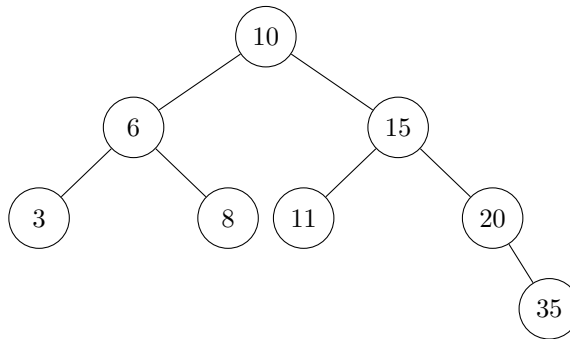
The tree is balanced.

Step 8: Insert 35

Since

$$35 > 10, \quad 35 > 15, \quad 35 > 20$$

35 is inserted to the right of 20.



Balance factors:

$$BF(35) = 0$$

$$BF(20) = 0 - 1 = -1$$

$$BF(15) = 1 - 2 = -1$$

$$BF(10) = 2 - 3 = -1$$

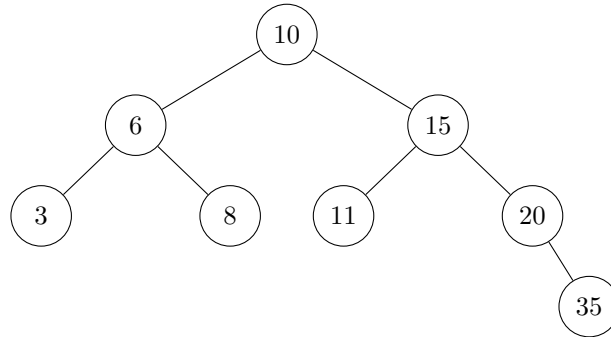
All balance factors satisfy:

$$-1 \leq BF \leq 1$$

Therefore, **no rotation is required**.

Final AVL Tree

The final AVL tree is:



FinalAVLTree

The final balance factors are:

$$BF(3) = 0, \quad BF(8) = 0, \quad BF(6) = 0$$

$$BF(11) = 0, \quad BF(35) = 0, \quad BF(20) = -1$$

$$BF(15) = -1, \quad BF(10) = -1$$

Hence,

$$-1 \leq BF \leq 1$$

for every node.

Therefore, the final tree is a valid AVL tree.

article tikz

AVL Tree Deletion Example

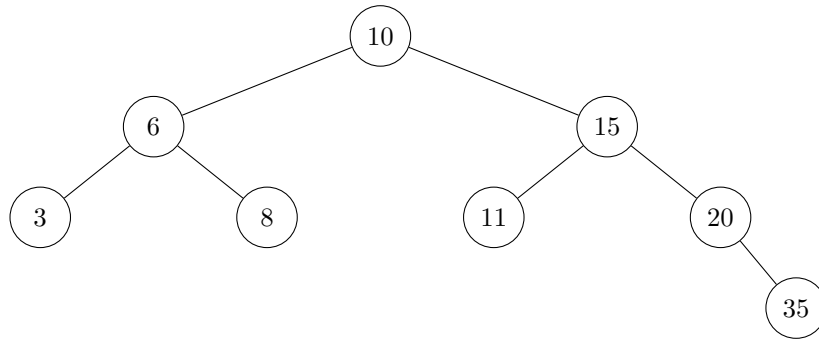
Deletionsequence :11, 20, 6

Balance Factor:

$$BF = h_L - h_R$$

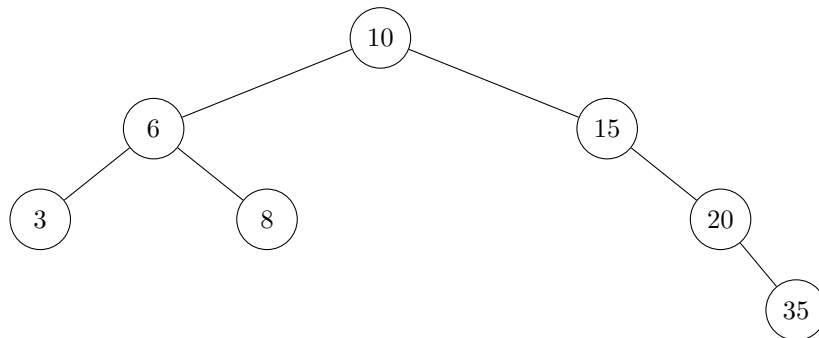
$$-1 \leq BF \leq 1$$

Initial AVL Tree



Step 1: Delete 11

11 is a leaf node, so remove it.



At node 15:

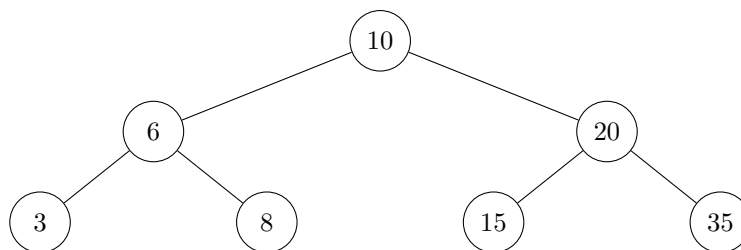
$$BF(15) = -2$$

At node 20:

$$BF(20) = -1$$

Therefore this is an **RR case**.

Perform a **Left Rotation at 15**.



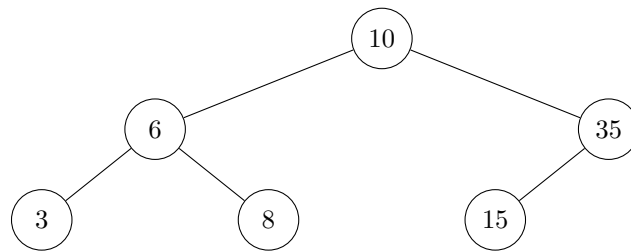
Step 2: Delete 20

20 has two children.

Its inorder successor is:

35

Replace 20 by 35 and delete the original 35.



$$BF(35) = 1$$

$$BF(10) = 0$$

Therefore, **no rotation is required**.

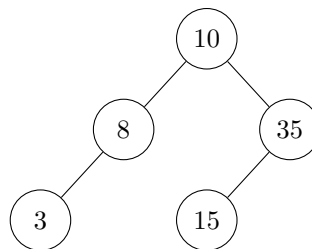
Step 3: Delete 6

6 has two children.

Its inorder successor is:

8

Replace 6 by 8 and delete the original 8.



Balance factors:

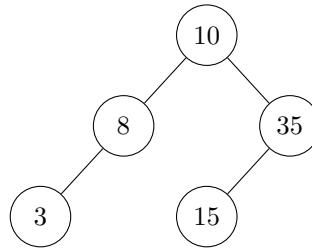
$$BF(8) = 1$$

$$BF(35) = 1$$

$$BF(10) = 0$$

Therefore, **no rotation is required**.

Final AVL Tree



FinalAVLTree

Red-Black Tree

A Red-Black Tree is a self-balancing binary search tree in which every node is colored red or black, and a set of rules involving these colors is used to keep the tree approximately balanced.

Rules for Red-Black Tree Construction

- (a) Every node is either RED or BLACK.
- (b) The root is always BLACK.
- (c) Every NULL leaf is considered BLACK.
- (d) A RED node cannot have a RED child.
- (e) Every path from a node to its descendant NULL leaves has the same number of BLACK nodes.
- (f) When insertion violates a rule, fix the tree using: Recoloring , Left rotation , Right rotation

Rules for Red-Black Tree Deletion

- (a) First, delete the node using the normal Binary Search Tree (BST) deletion procedure.
- (b) If the deleted node is **RED**, no further fixing is usually required.
- (c) If the deleted node is **BLACK**, a double-black condition may occur and the tree must be fixed.
- (d) If the deleted black node has one **RED** child, replace the deleted node with its child and color the child **BLACK**.
- (e) If a double-black condition occurs, consider the color of the sibling and its children.
- (f) If the sibling is **RED**:
 - Recolor the sibling **BLACK**.
 - Recolor the parent **RED**.
 - Perform a rotation around the parent.
 - Continue with the appropriate deletion case.
- (g) If the sibling is **BLACK** and both of its children are **BLACK**:
 - Color the sibling **RED**.
 - Move the double-black condition to the parent.
- (h) If the sibling is **BLACK**, the near child is **RED**, and the far child is **BLACK**:
 - Recolor the near child **BLACK**.
 - Recolor the sibling **RED**.
 - Perform a rotation around the sibling.
 - This converts the situation into the next case.
- (i) If the sibling is **BLACK** and the far child is **RED**:
 - Give the sibling the color of the parent.
 - Color the parent **BLACK**.
 - Color the far child **BLACK**.
 - Perform a rotation around the parent.
 - The double-black condition is removed.
- (j) Finally, ensure that the root of the Red-Black Tree is **BLACK**.

Red-Black Tree Insertion Example

Insert the elements in the following order:

3, 6, 8, 10, 11, 15, 20, 35

Red-Black Tree Insertion Rules

- (a) Every node is either RED or BLACK.
- (b) The root is always BLACK.
- (c) Every NULL leaf is BLACK.
- (d) A RED node cannot have a RED child.
- (e) Every path from a node to its descendant NULL leaves contains the same number of BLACK nodes.

Color Definitions

- RED node: red circle with white text.
- BLACK node: black circle with white text.

Step 1: Insert 3

The first node becomes the root and is colored BLACK.



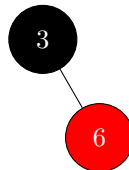
Root3 = BLACK

The tree is valid.

Step 2: Insert 6

Since $6 > 3$, insert 6 as the right child of 3.

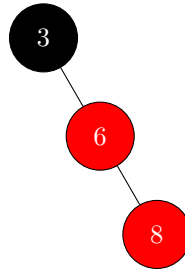
New nodes are inserted as RED.



The root is BLACK and there is no RED-RED violation.

Step 3: Insert 8

Since $8 > 6 > 3$, insert 8 as the right child of 6.



Nodes 6 and 8 are both RED.

This violates the Red-Black Tree rule.

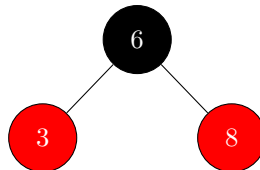
Case : RR

Perform a LEFT ROTATION at node 3.

Recolor:

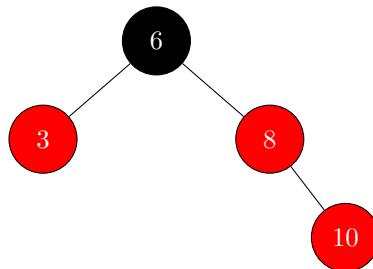
$6 = BLACK, \quad 3 = RED$

After rotation:



Step 4: Insert 10

Since $10 > 6$ and $10 > 8$, insert 10 as the right child of 8.



Nodes 8 and 10 are both RED.

The uncle of 10 is node 3, which is also RED.

Therefore, recolor:

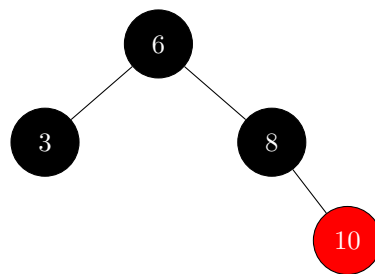
$$3 = \textit{BLACK}$$

$$8 = \textit{BLACK}$$

$$6 = \textit{RED}$$

The root must always be BLACK, so recolor 6 back to BLACK.

After recoloring:

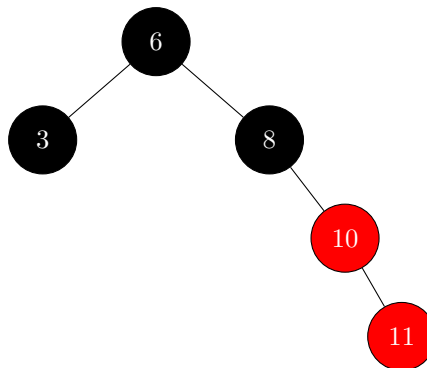


Step 5: Insert 11

Since

$$11 > 6, \quad 11 > 8, \quad 11 > 10$$

insert 11 as the right child of 10.



Nodes 10 and 11 are both RED.

The uncle of 11 is node 8, which is BLACK.

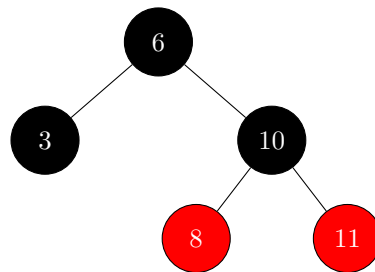
This is an RR case.

Perform a LEFT ROTATION at node 8.

Recolor:

$$10 = \text{BLACK}, \quad 8 = \text{RED}$$

After rotation:

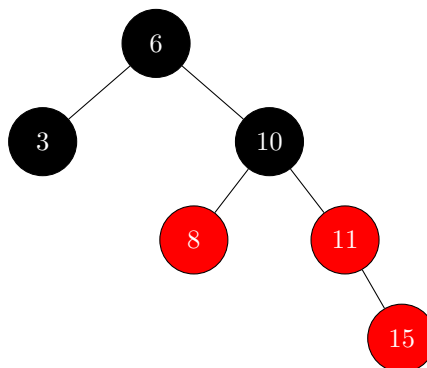


Step 6: Insert 15

Since

$$15 > 6, \quad 15 > 10, \quad 15 > 11$$

insert 15 as the right child of 11.



Nodes 11 and 15 are both RED.

The uncle of 15 is node 8, which is RED.

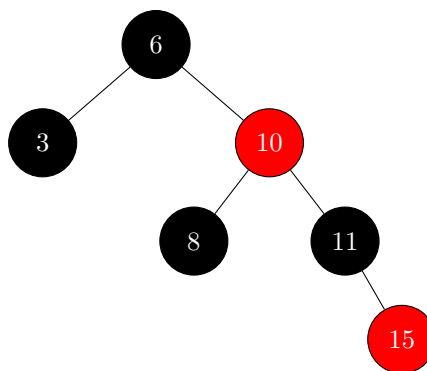
Therefore, recolor:

$11 = \textit{BLACK}$

$8 = \textit{BLACK}$

$10 = \textit{RED}$

After recoloring:

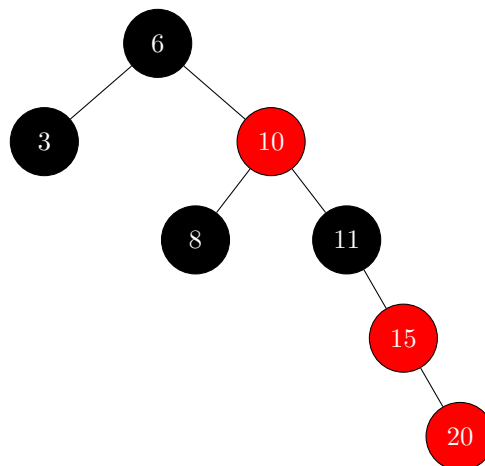


Step 7: Insert 20

Since

$$20 > 6, \quad 20 > 10, \quad 20 > 11, \quad 20 > 15$$

insert 20 as the right child of 15.



Nodes 15 and 20 are both RED.

The uncle of 20 is node 11, which is BLACK.

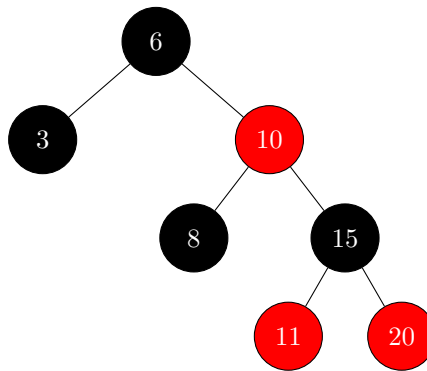
This is an RR case.

Perform a LEFT ROTATION at node 11.

Recolor:

$$15 = \textit{BLACK}, \quad 11 = \textit{RED}$$

After rotation:

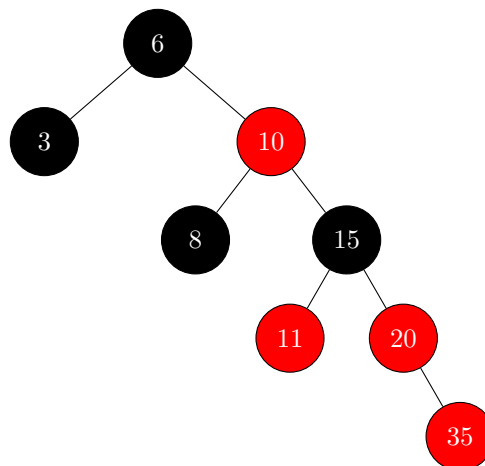


Step 8: Insert 35

Since

$$35 > 6, \quad 35 > 10, \quad 35 > 15, \quad 35 > 20$$

insert 35 as the right child of 20.



Nodes 20 and 35 are both RED.

The uncle of 35 is node 11, which is RED.

Therefore, recolor:

$$20 = BLACK$$

$$11 = BLACK$$

$$15 = RED$$

Now nodes 10 and 15 are both RED.

The uncle of 15 is node 8, which is BLACK.

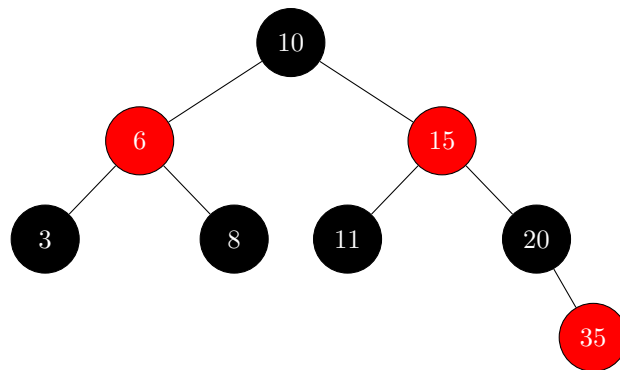
This is an RR case.

Perform a LEFT ROTATION at node 6.

Recolor:

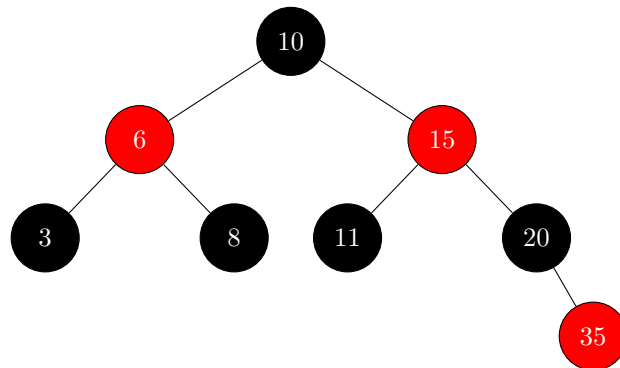
$$10 = BLACK, \quad 6 = RED$$

After rotation:



Final Red-Black Tree

The final tree is:



FinalRed – BlackTree

Final Node Colors

$10 = \text{BLACK}$

$6 = \text{RED}, \quad 15 = \text{RED}$

$3 = \text{BLACK}, \quad 8 = \text{BLACK}, \quad 11 = \text{BLACK}$

$20 = \text{BLACK}, \quad 35 = \text{RED}$

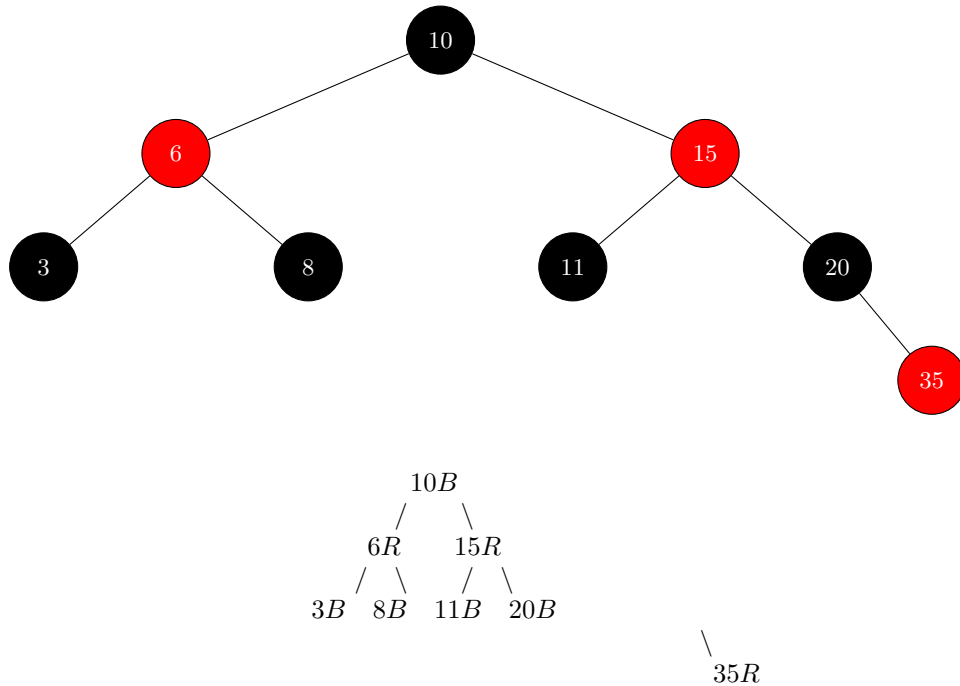
The final tree satisfies all Red-Black Tree properties.
article tikz

Red-Black Tree Deletion Example

Deletionsequence : $11 \rightarrow 20 \rightarrow 6$

B = Black, **R = Red**

Initial Red-Black Tree



Step 1: Delete 11

11 is a BLACK leaf.

Therefore, a double-black condition occurs.

The sibling is 20B and its far child is 35R.

This is **Case 4**.

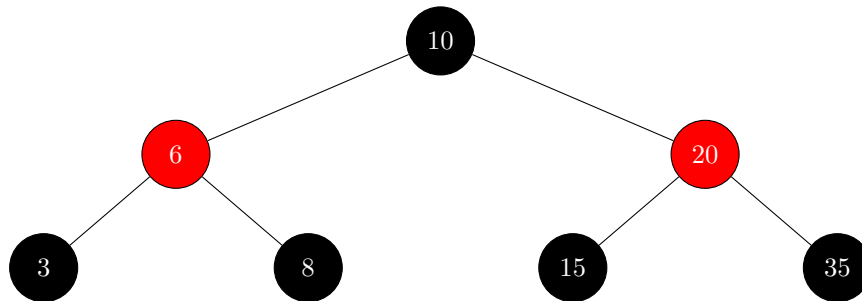
- 20 takes the color of parent 15.
- 15 becomes BLACK.
- 35 becomes BLACK.
- Perform a LEFT rotation at 15.

$$20B \rightarrow 20R$$

$$15R \rightarrow 15B$$

$$35R \rightarrow 35B$$

After deleting 11



Step 2: Delete 20

20 has two children: 15 and 35.

Its inorder successor is:

35

Replace 20 with 35 and delete the original 35.

The deleted node was BLACK, so a double-black condition occurs.

The sibling is 15B.

Both children of 15 are BLACK/NIL.

Therefore, this is **Case 2**.

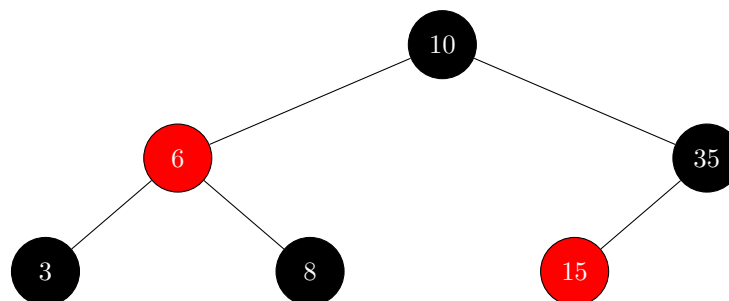
$15B \rightarrow 15R$

The double-black moves to 35.

The replacement node 35 becomes BLACK.

No rotation is required.

After deleting 20



Step 3: Delete 6

6 has two children:

3 and 8

Its inorder successor is:

8

Replace 6 with 8 and delete the original 8.

The original 8 was BLACK, so a double-black condition occurs.

The sibling is 3B.

Both children of 3 are BLACK/NIL.

Therefore, this is **Case 2**.

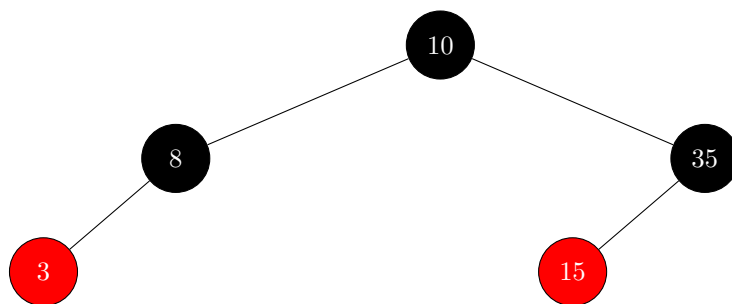
$3B \rightarrow 3R$

The double-black moves to 8.

Since 8 is the replacement for the deleted black node, it becomes BLACK.

No rotation is required.

Final Red-Black Tree



FinalTree

```
      10B
     /  \
    8B   35B
    /    \
   3R    15R
```

B Tree

A B-Tree is a self-balancing multiway search tree in which each node can contain multiple keys and multiple children. It is designed to keep the tree balanced and reduce the number of disk accesses during searching, insertion, and deletion. B-Trees are widely used in databases and file systems.

Properties of B-Tree of order m

- (a) Each node can have at most m children.
- (b) Each node can contain at most $m - 1$ keys.
- (c) Except for the root, every internal node has atleast $m/2$ children.
- (d) The root has at least 2 children if it is not a leaf.
- (e) All leaf nodes are at the same level.
- (f) Keys within each node are stored in sorted order.
- (g) A node with k keys has $k + 1$ children if it is an internal node.
- (h) Searching, insertion, and deletion take

$$O(\log n)$$

time.

B+ Tree

A B+ Tree is a balanced multiway search tree and an extension of the B-Tree in which internal nodes contain only keys for indexing, while all actual data records are stored in the leaf nodes. The leaf nodes are usually connected using pointers, allowing efficient sequential and range searching. B+ Trees are widely used in database indexing and file systems.

Properties of B+ Tree of order m

- (a) The internal nodes contain search keys and child pointers.
- (b) The actual records/data are stored only in the leaf nodes.
- (c) All leaf nodes are at the same level.
- (d) The leaf nodes are connected using linked pointers.
- (e) The keys in the leaf nodes are stored in a sorted order.

- (f) The internal nodes act as an index to locate the required leaf.
Searching, insertion, and deletion take

$$O(\log n)$$

time.

- (g) B+ Trees are particularly efficient for range queries and sequential access.